Chapter 10 | Worked solutions with Python

Incremental Decoding and KV Representations

Goal

Work through all 12 exercises. Start with the timeline of one prediction, then count saved computation and stored vectors. Use exact array checks to understand chunk masks and shared heads. Finally, move a linear projection without changing the attention calculation. Exercises 9-12 are optional. The written edition retains every explanation and plot; the notebook PDF also includes all code and saved results.

Setup

Open this notebook inside `transformers-book`. Install the repository's `requirements.txt`, plus `matplotlib` and `jupyterlab`, and run all cells in order on CPU. No model or dataset download is needed. Counts and cache copies are checked exactly; floating-point comparisons declare their tolerances. The saved execution uses Python 3.12 and NumPy 2.4.

In [1]

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch10_kv" / "model.py"
    )
    if model_path.is_file():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

Steps

1. Follow the first two predictions

Prefill consumes the prompt at positions 0 and 1. The vocabulary logits at **red**, position 1, predict **car**. Sampling selects an ID; it does not run that ID through the decoder. The next call consumes car at absolute position 2, even though its local input array has only row zero.

Call	Input positions	Cache positions after call	Row used to predict	Sampled ID
Prefill	0: the, 1: red	0, 1	1	car
First append	2: car	0, 1, 2	2	stopped

Each layer appends car's key and value before its query reads keys 0, 1, and 2. Including its own position is causal: that input is already known. The query determines a fresh weighted sum of the three values. Every layer retains its own keys and values for those three positions. Earlier queries and attention weights are not needed as history for the next query.

If we stop after sampling stopped, it has **no cache entry**. It enters the cache only if another call consumes it at position 3. Two sampled outputs therefore require prefill and one append, not two appends. This convention also matters when reporting decoding time.

In [2]

```
prompt_length, sampled_outputs = 2, 2
append_calls = sampled_outputs - 1
final_cache_length = prompt_length + append_calls
assert append_calls == 1 and final_cache_length == 3
print("Calls after prefill:", append_calls)
print("Cached positions:", list(range(final_cache_length)))
```

Output

```
Calls after prefill: 1
Cached positions: [0, 1, 2]
```

2. Count rows and comparisons

The prompt has three positions. The next two input calls see prefixes of length four and five. A full causal pass evaluates one query at every position; at length n, the allowed rows contain $1, 2, \ldots, n$ pairs. A cached call evaluates just the last query, which still reads all n keys.

Prefix length	Full rows	Cached rows	Full allowed pairs	Cached pairs
4	4	1	10	4
5	5	1	15	5
Total after prefill	9	2	25	9

The cache removes repeated queries and projections, but the new query still compares with every allowed key. For P prompt positions and M subsequent one-token inputs, the cached pair count is

$$\sum_{j=1}^{M}(P+j) = MP + \frac{M(M+1)}{2}.$$

Here that gives $2(3) + 2(3)/2 = 9$. These are allowed pairs per head and layer, not total FLOPs or a measured speedup. A dense score kernel may also compute masked entries. Prefill supplies the first prediction, so these two later calls bring the number of sampled outputs to three.

In [3]

```
lengths = [4, 5]
full_rows = sum(lengths)
full_pairs = sum(n * (n + 1) // 2 for n in lengths)
cached_rows, cached_pairs = len(lengths), sum(lengths)
assert (full_rows, full_pairs, cached_rows, cached_pairs) == (9, 25, 2, 9)
print("Full rows / pairs:", full_rows, full_pairs)
print("Cached rows / pairs:", cached_rows, cached_pairs)
print("Sampled outputs including prefill:", 1 + cached_rows)
```

Output

```
Full rows / pairs: 9 25
Cached rows / pairs: 2 9
Sampled outputs including prefill: 3
```

3. Ask two questions of one history

The keys form the identity matrix. Query $(1, 0)$ therefore gives dot products $(1, 0)$, and query $(0, 1)$ gives $(0, 1)$. Divide by $\sqrt{2}$ before softmax. Let $s = 1/\sqrt{2}$ and $a = e^s/(e^s + 1) \approx 0.669762$. The weights are $(a, 1 - a)$ and $(1 - a, a)$.

Query	Scaled scores	Output from the shared values
$(1, 0)$	$(s, 0)$	$(2a, 1 - a) \approx (1.339523, 0.330238)$
$(0, 1)$	$(0, s)$	$(2(1 - a), a) \approx (0.660477, 0.669762)$

Both heads see exactly the same values, $(2, 0)$ and $(0, 1)$. Their queries choose different mixing coefficients. Sharing the source information does not force the heads to ask the same question or produce the same output. The notebook calls the canonical grouped-attention helper with two query heads and one KV head.

In [4]

```
from examples.ch10_kv.model import grouped_attention
queries = np.array([[[[1., 0.]], [[0., 1.]]]])
keys = np.eye(2)[None, None]
values = np.array([[[[2., 0.], [0., 1.]]]])
outputs, weights = grouped_attention(queries, keys, values)
a = np.exp(1 / np.sqrt(2)) / (1 + np.exp(1 / np.sqrt(2)))
expected = [[2*a, 1-a], [2*(1-a), a]]
np.testing.assert_allclose(outputs[0, :, 0], expected, rtol=0, atol=1e-14)
print("Weights:", weights[0, :, 0].round(6))
print("Outputs:", outputs[0, :, 0].round(6))
```

Output

```
Weights: [[0.669762 0.330238]
 [0.330238 0.669762]]
Outputs: [[1.339523 0.330238]
 [0.660477 0.669762]]
```

4. Count a decoder's cache

One KV head stores $64 + 64 = 128$ coordinates per position per layer. At two bytes per coordinate that is 256 bytes. Multiply by both sequences, all 1,024 positions, and all 12 layers:

$$2 \cdot 1024 \cdot 12 \cdot (64 + 64) \cdot 2 = 6291456 \text{ bytes} = 6 \text{ MiB}.$$

The first factor of two counts sequences; the last counts bytes per scalar. A MiB contains 2^{20} bytes. Multiplying the one-head total gives:

KV heads	Total bytes	Total MiB	Query heads per KV head
12	75497472	72	1
3	18874368	18	4
1	6291456	6	12

All three choices retain 12 query heads. The persistent cache counts stored KV heads, not queries. Model weights, scores, and temporary buffers are separate. With unequal valid lengths, packed storage uses their sum; padded storage may reserve the longest length for every batch item.

In [5]

```
from examples.ch10_kv.model import cache_bytes
for heads, expected_mib in [(12, 72), (3, 18), (1, 6)]:
    size = cache_bytes(2, 12, 1024, heads, 64, 64, 2)
    assert size == expected_mib * 2**20
    print(heads, "KV heads:", size, "bytes;", size // 2**20,
          "MiB;", 12 // heads, "queries per group")
```

Output

```
12 KV heads: 75497472 bytes; 72 MiB; 1 queries per group
3 KV heads: 18874368 bytes; 18 MiB; 4 queries per group
1 KV heads: 6291456 bytes; 6 MiB; 12 queries per group
```

5. Build a mask for a chunk

The cache holds keys 0 and 1. Append two new keys at positions 2 and 3, one for each of the two new queries. The query rows have local indices 0 and 1 in the chunk array, but their **absolute positions**, meaning their indices in the whole sequence, are 2 and 3. Those whole-sequence indices determine which keys each query can read.

A **causal mask** is a Boolean array that allows a query to read a key only if the key is at the same position or an earlier one. `causal_mask` takes the absolute position of the first query, the number of queries, and the number of keys. Here the correct arguments are 2, 2, and 4. In both runs, columns are keys 0, 1, 2, and 3 in that order. Convert Boolean entries to integers for display so that 1 means an allowed read and 0 means a blocked read.

In [6]

```
from examples.ch10_kv.model import causal_mask

correct_mask = causal_mask(2, 2, 4)
reset_mask = causal_mask(0, 2, 4)
lost_reads = correct_mask & ~reset_mask

np.testing.assert_array_equal(
    correct_mask, [[1, 1, 1, 0], [1, 1, 1, 1]]
)
np.testing.assert_array_equal(
    reset_mask, [[1, 0, 0, 0], [1, 1, 0, 0]]
)

print("Correct mask for absolute queries 2 and 3")
print(correct_mask.astype(int))
print("Mask after assigning the queries positions 0 and 1")
print(reset_mask.astype(int))

for row, abs_query in enumerate((2, 3)):
    key_ids = np.flatnonzero(lost_reads[row]).tolist()
    print(f"Actual query {abs_query} loses keys {key_ids}")
```

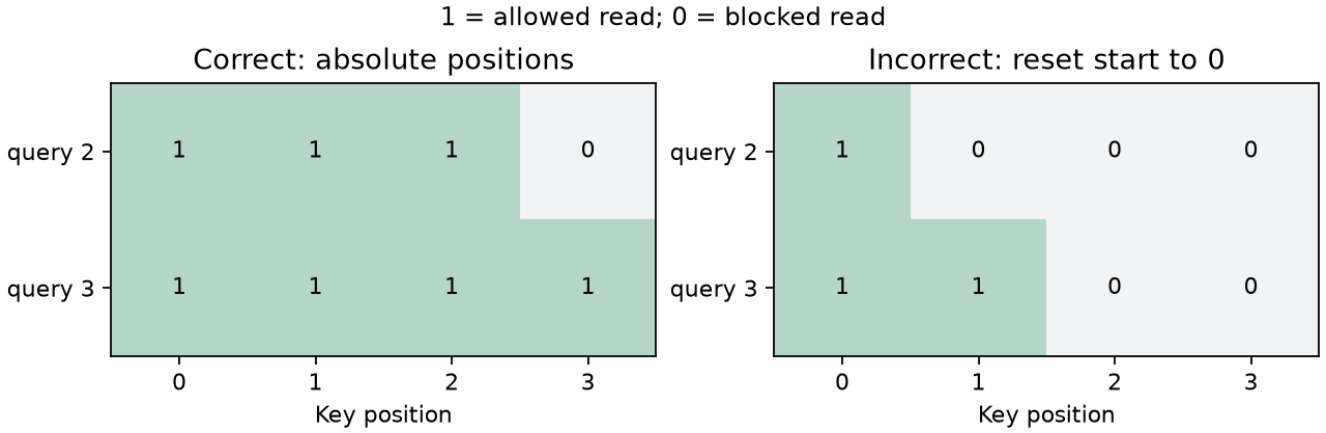
Output

```
Correct mask for absolute queries 2 and 3
[[1 1 1 0]
 [1 1 1 1]]
Mask after assigning the queries positions 0 and 1
[[1 0 0 0]
 [1 1 0 0]]
Actual query 2 loses keys [1, 2]
Actual query 3 loses keys [2, 3]
```

In [7]

```
import io
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from IPython.display import Image, display

fig, axes = plt.subplots(1, 2, figsize=(8, 2.6), constrained_layout=True)
for ax, mask, title in zip(axes, (correct_mask, reset_mask),
                           ("Correct: absolute positions", "Incorrect: reset start to 0")):
    ax.imshow(mask, cmap=ListedColormap(["#f1f3f5", "#b4d5c8"]), vmin=0, vmax=1)
    ax.set(xticks=range(4), yticks=range(2), yticklabels=["query 2", "query 3"],
           xlabel="Key position", title=title)
    for (row, col), allowed in np.ndenumerate(mask):
        ax.text(col, row, str(int(allowed)), ha="center", va="center")
fig.suptitle("1 = allowed read; 0 = blocked read", fontsize=11)
image_buffer = io.BytesIO()
fig.savefig(image_buffer, format="png", dpi=160)
plt.close(fig)
display(Image(data=image_buffer.getvalue()))
```



Query 2 should read keys 0, 1, and 2. The reset makes the mask treat it as query 0, so it loses keys 1 and 2. Query 3 should read all four keys. The reset makes it look like query 1, so it loses keys 2 and 3. The data can still contain the correct keys in their correct order. The mask decides which of those stored keys the new queries actually get to use.

With two sequences and four query heads, the scores have shape $[2, 4, 2, 4]$ in batch, head, query, and key order. Give the mask shape $[2, 1, 2, 4]$. Broadcasting uses the single head entry for all four heads of a sequence. The two batch entries still carry independent patterns. A mask of shape $[2, 2, 4]$ would align its leading axis with the head axis because NumPy aligns dimensions from the right.

For a concrete example, let sequence A use queries at positions 2 and 3, and sequence B use queries at positions 1 and 2. Each has four available key positions in its key array. Construct the two causal patterns, then insert an axis of length one at the head position. The broadcast view below shows how the mask aligns with the scores in `grouped_attention`.

In [8]

```
pattern_a = correct_mask
pattern_b = causal_mask(1, 2, 4)
batch_mask = np.stack((pattern_a, pattern_b))[:, None, :, :]
score_mask = np.broadcast_to(batch_mask, (2, 4, 2, 4))

np.testing.assert_array_equal(score_mask[0, 3], pattern_a)
np.testing.assert_array_equal(score_mask[1, 3], pattern_b)
assert not np.array_equal(score_mask[0, 3], score_mask[1, 3])

print("Mask supplied to attention:", batch_mask.shape)
print("Mask aligned with scores:", score_mask.shape)
for sequence in range(2):
    print(f"Sequence {sequence}, head 3")
    print(score_mask[sequence, 3].astype(int))
```

Output

```
Mask supplied to attention: (2, 1, 2, 4)
Mask aligned with scores: (2, 4, 2, 4)
Sequence 0, head 3
[[1 1 1 0]
 [1 1 1 1]]
Sequence 1, head 3
[[1 1 0 0]
 [1 1 1 0]]
```

Sequence B's first query is one position earlier than A's and cannot read key 2. B's second query cannot read key 3. Within either sequence, every head receives the same permission table. If an input also had padding, intersect its causal pattern with a mask for valid keys. Every query sent to this implementation needs at least one allowed key. This example constructs a mask for attention on supplied projections. The teaching cache API assumes equally long, contiguous histories with one common chunk start.

6. Compare full and cached attention

Take the same six positions and evaluate them in two ways. The full pass supplies all six query rows at once. The chunked pass supplies the first three rows, then two more, then the last row. The input projections, head grouping, and causal rule stay fixed. This lets us isolate the effect of splitting the work into chunks.

`projected_example()` supplies already projected query, key, and value arrays. Their axes are batch, head, position, and feature. Keep four query heads but slice the **head axis** of the keys and values using `[:, :2]`. That preserves all six positions for the two retained KV heads. Query heads 0 and 1 read KV head 0. Query heads 2 and 3 read KV head 1. This is grouped-query attention. The full pass uses a six-by-six causal mask to block later keys, even though they are already present in the arrays.

The returned weights have one entry per batch item, query head, query row, and key position. The notebook prints the full matrix of weights for the first query head. Each row adds to one, and every entry for a future key is zero. The outputs have two features per head and query because the value vectors have width two.

In [9]

```
from examples.ch10_kv.model import grouped_attention
from examples.ch10_kv.run_examples import projected_example

q, all_keys, all_values = projected_example()
k, v = all_keys[:, :2], all_values[:, :2]
full_mask = causal_mask(0, 6, 6)
full_out, full_weights = grouped_attention(q, k, v, full_mask)

assert q.shape == full_out.shape == (1, 4, 6, 2)
assert k.shape == v.shape == (1, 2, 6, 2)
assert full_weights.shape == (1, 4, 6, 6)
assert np.all(full_weights[..., ~full_mask] == 0)
np.testing.assert_allclose(
    full_weights.sum(axis=-1), 1, rtol=0, atol=1e-12
)

print("Query shape:", q.shape)
print("Key and value shape:", k.shape)
print("Full output shape:", full_out.shape)
print("Full weight shape:", full_weights.shape)
print("Full causal weights for head 0")
with np.printoptions(precision=6, suppress=True):
    print(full_weights[0, 0])
```

Output

```
Query shape: (1, 4, 6, 2)
Key and value shape: (1, 2, 6, 2)
Full output shape: (1, 4, 6, 2)
Full weight shape: (1, 4, 6, 6)
Full causal weights for head 0
[[1.       0.       0.       0.       0.       0.      ]
 [0.232691 0.767309 0.       0.       0.       0.      ]
 [0.274329 0.408696 0.316974 0.       0.       0.      ]
 [0.446164 0.2095   0.217144 0.127192 0.       0.      ]
 [0.212473 0.090016 0.167331 0.406598 0.123582 0.      ]
 [0.076939 0.148178 0.14785  0.253259 0.247774 0.126   ]]
```

In the chunked run, `run_chunks` calls `attention_step` once for each chunk. Each call keeps the previous keys and values, appends the new ones on the position axis, and uses the previous cache length as the absolute query start.

Chunk	Previous cache length	Absolute queries	Key positions after append
1	0	0, 1, 2	0, 1, 2
2	3	3, 4	0, 1, 2, 3, 4
3	5	5	0, 1, 2, 3, 4, 5

In the second chunk, query 3 can read keys 0 through 3 and query 4 can read keys 0 through 4. Key 4 has already been appended when query 3 runs. It still gets zero weight in that row. In the full pass the same row can see the same four keys, while its entries for keys 4 and 5 are masked. This reasoning applies to every query, so each head forms the same weighted sum of the same allowed values, up to rounding.

Concatenate the chunk outputs in position order, then compare each float64 output coordinate with the corresponding full-pass coordinate. The relative tolerance is zero. The absolute tolerance of 10^{-12} applies equally to outputs near zero and to larger outputs. The cache comparison is exact because storing or appending a projected row copies its entries without recalculating them.

In [10]

```
from examples.ch10_kv.run_examples import run_chunks

chunk_sizes = (3, 2, 1)
cached_out, (cached_k, cached_v) = run_chunks(q, k, v, chunk_sizes)
np.testing.assert_allclose(
    cached_out, full_out, rtol=0, atol=1e-12
)
np.testing.assert_array_equal(cached_k, k)
np.testing.assert_array_equal(cached_v, v)

error = np.abs(cached_out - full_out)
start = 0
for size in chunk_sizes:
    end = start + size
    local_error = error[:, :, start:end].max()
    print(f"Positions {start} through {end - 1}: {local_error:.3e}")
    start = end

print(f"Maximum output difference: {error.max():.3e}")
print("Cache shapes:", cached_k.shape, cached_v.shape)
print("Final query output for each head in chunked attention")
with np.printoptions(precision=9, suppress=True):
    print(cached_out[0, :, -1, :])
```

Output

```
Positions 0 through 2: 0.000e+00
Positions 3 through 4: 0.000e+00
Positions 5 through 5: 1.665e-16
Maximum output difference: 1.665e-16
Cache shapes: (1, 2, 6, 2) (1, 2, 6, 2)
Final query output for each head in chunked attention
[[0.149017163 0.312294206]
 [0.543017043 0.847100077]
 [0.066147024 0.367019286]
 [0.353063196 0.351276429]]
```

The maximum absolute difference is about $1.67 \cdot 10^{-16}$ for these arrays, well below 10^{-12}. Both output arrays have shape $[1, 4, 6, 2]$. Each cache array has shape $[1, 2, 6, 2]$ and stores exactly the projected rows for its two KV heads. The output uses four heads because the four separate queries can produce different weight patterns, even when two of them read the same history. The cache holds two heads because the history is shared within each group.

This check isolates the effect of processing the same attention inputs in chunks. Passing both computations the same projected arrays keeps any differences in upstream positional transforms or other decoder layers outside the experiment.

Equality between two implementations is useful, but both could share a mask bug. Change only the value at position 4. Causality requires positions 0-3 to stay unchanged, because that value lies in their future. The query at position 4 can read it, so the test also checks that the perturbation actually matters. The largest earlier-output change is exactly zero; the change at position 4 is about 39.056.

In [11]

```
changed_v = v.copy()
changed_v[:, :, 4] += 100
changed_out, _ = run_chunks(q, k, changed_v, (3, 2, 1))
np.testing.assert_allclose(changed_out[:, :, :4], cached_out[:, :, :4],
                           rtol=0, atol=1e-12)
assert np.max(np.abs(changed_out[:, :, 4] - cached_out[:, :, 4])) > 1
print("Largest earlier change:", np.max(np.abs(changed_out[:, :, :4] - cached_out[:, :, :4])))
print("Change at position 4:", np.max(np.abs(changed_out[:, :, 4] - cached_out[:, :, 4])))
```

Output

```
Largest earlier change: 0.0
Change at position 4: 39.05598999806227
```

7. Check what a byte count means

Take the same six projected positions as in Exercise 6 and replay the chunk schedule $(3, 2, 1)$. Keep all four query heads and two key and value heads. Two query heads read each KV head. The cache therefore stores two heads per position in each of its arrays.

Each cached array has shape $[1, 2, 6, 2]$, in batch, head, position, and feature order. It contains $1 \cdot 2 \cdot 6 \cdot 2 = 24$ scalars. NumPy calls the number of bytes per scalar `itemsize`. With `float64`, each scalar uses eight bytes, so each array uses $24 \cdot 8 = 192$ bytes. The two arrays together use 384 bytes.

The helper `cache_bytes` counts the two feature widths together. Here we pass $B = 1$ batch item, $L = 1$ layer, $T = 6$ positions, $H_{KV} = 2$ KV heads, and $d_k = d_v = 2$. With s bytes per scalar, the formula is

$$BLTH_{KV}(d_k + d_v)s = 1 \cdot 1 \cdot 6 \cdot 2 \cdot (2 + 2) \cdot 8 = 384 \text{ bytes.}$$

We will compare the formula with the sum of `nbytes` from the two arrays returned by `run_chunks`. Then we will replay the same chunks after casting the keys and values to `float32`, with the queries unchanged. Each array still has 24 scalars. At four bytes each, those scalars take 96 bytes per array and 192 bytes in total.

In [12]

```
from examples.ch10_kv.model import cache_bytes
from examples.ch10_kv.run_examples import projected_example, run_chunks

byte_queries, byte_keys, byte_values = projected_example()
byte_keys, byte_values = byte_keys[:, :2], byte_values[:, :2]
byte_chunks = (3, 2, 1)
byte_results = {}

for dtype in (np.float64, np.float32):
    keys = byte_keys.astype(dtype, copy=False)
    values = byte_values.astype(dtype, copy=False)
    output, (stored_keys, stored_values) = run_chunks(
        byte_queries, keys, values, byte_chunks
    )
    byte_results[dtype] = output, stored_keys, stored_values

    batch, heads, positions, key_width = stored_keys.shape
    value_width = stored_values.shape[-1]
    itemsize = stored_keys.itemsize
    counted = stored_keys.nbytes + stored_values.nbytes
    predicted = cache_bytes(
        batch, 1, positions, heads, key_width, value_width, itemsize
    )

    np.testing.assert_array_equal(stored_keys, keys)
    np.testing.assert_array_equal(stored_values, values)
    assert stored_keys.dtype == stored_values.dtype == np.dtype(dtype)
    assert output.dtype == np.dtype(np.float64)
    assert counted == predicted == 48 * itemsize

    print(f"{np.dtype(dtype).name} storage ({itemsize} bytes/scalar)")
    print(f"  keys    {stored_keys.dtype}  {stored_keys.nbytes} B")
    print(f"  values  {stored_values.dtype}  {stored_values.nbytes} B")
    print(f"  total   {counted} B (formula {predicted} B)")
    print(f"  attention output dtype {output.dtype}")
```

Output

```
float64 storage (8 bytes/scalar)
  keys    float64  192 B
  values  float64  192 B
  total   384 B (formula 384 B)
  attention output dtype float64
float32 storage (4 bytes/scalar)
  keys    float32  96 B
  values  float32  96 B
  total   192 B (formula 192 B)
  attention output dtype float64
```

Both runs produce `float64` attention output. The stored arrays keep the dtype of the rows appended to them. Inside `grouped_attention`, the queries are cast to `float64` before the score multiplication, and softmax produces `float64` weights. NumPy promotes the `float32` keys and values for those respective multiplications. For example, the first key scalar was about 0.60587896545. Storing it as `float32` rounds it to about 0.60587894917. Promoting the stored scalar to `float64` for arithmetic keeps that rounded value.

To isolate the effect of storage precision, cast the rounded keys and values back to `float64` before replaying. That replay should match the run that stored `float32`. Comparing either with the original `float64` run measures the effect of rounding the inputs for this example.

In [13]

```
out64, key64, _ = byte_results[np.float64]
out32, key32, value32 = byte_results[np.float32]
promoted_output, _ = run_chunks(
    byte_queries,
    key32.astype(np.float64),
    value32.astype(np.float64),
    byte_chunks,
)

np.testing.assert_allclose(out32, promoted_output, atol=1e-12, rtol=0)
print(f"Original key scalar  {key64.flat[0]:.12f}")
print(f"Rounded key scalar   {float(key32.flat[0]):.12f}")
delta = np.max(np.abs(out64 - out32))
print(f"Max output change from rounding  {delta:.3e}")
```

Output

```
Original key scalar  0.605878965449
Rounded key scalar   0.605878949165
Max output change from rounding  5.744e-08
```

The smaller cache changes the result by about $5.74 \cdot 10^{-8}$ at the most affected output coordinate in this run. That number depends on the input and the calculation. The byte totals are exact. `nbytes` measures the payload of the two arrays. It leaves out the queries, attention scores and weights, allocations used when appending a chunk, and the surrounding Python objects.

8. Compare two orders of latent attention

Imagine one query reading two earlier positions. In the expanded calculation, we apply each head's key and value matrices to both cached rows first. In the latent calculation, each head transforms its content query once, compares it with the shared latent rows, and mixes those rows before applying the value matrix. The rotary part of each score uses the same positional vectors in both runs.

`latent_inputs()` supplies six arrays in the order expected by both functions. They are the content queries, rotary queries, latent history, rotary key history, key projections, and value projections. The `*inputs` syntax passes them as six positional arguments. The example gives one query per head for two heads. Each output row contains two value coordinates, and each weight row contains two probabilities, one per cached position. We compare both arrays at absolute tolerance 10^{-12} and zero relative tolerance.

In [14]

```
from examples.ch10_kv.latent import (
    expanded_attention, latent_attention, latent_inputs,
)

inputs = latent_inputs()
expanded_out, expanded_w = expanded_attention(*inputs)
latent_out, latent_w = latent_attention(*inputs)

np.testing.assert_allclose(expanded_w, latent_w, atol=1e-12, rtol=0)
np.testing.assert_allclose(expanded_out, latent_out, atol=1e-12, rtol=0)

with np.printoptions(precision=8):
    print("Weights (expanded, then latent)")
    print(expanded_w)
    print(latent_w)
    print("Outputs (expanded, then latent)")
    print(expanded_out)
    print(latent_out)
weight_delta = np.max(np.abs(expanded_w - latent_w))
output_delta = np.max(np.abs(expanded_out - latent_out))
print(f"Max weight delta {weight_delta:.3e}")
print(f"Max output delta {output_delta:.3e}")
```

Output

```
Weights (expanded, then latent)
[[0.81757448 0.18242552]
 [0.88079708 0.11920292]]
[[0.81757448 0.18242552]
 [0.88079708 0.11920292]]
Outputs (expanded, then latent)
[[1.18242552 3.2702979 ]
 [3.52318831 1.11920292]]
[[1.18242552 3.2702979 ]
 [3.52318831 1.11920292]]
Max weight delta 0.000e+00
Max output delta 0.000e+00
```

Reconstruct the scores for the first head, which is at index 0 in the arrays. Its queries are $q^C = (2, 1)$ for content and $q^R = (0, 1)$ for the rotary part. The content key matrix is $W_{UK} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$. Applying it to the two latent rows gives expanded content keys $(1, 2)W_{UK} = (3, 2)$ and $(2, 0)W_{UK} = (2, 0)$.

Contribution	Position 0	Position 1
Content $q^C \cdot k^C$	$2 \cdot 3 + 1 \cdot 2 = 8$	$2 \cdot 2 + 1 \cdot 0 = 4$
Rotary $q^R \cdot k^R$	$(0, 1) \cdot (1, 0) = 0$	$(0, 1) \cdot (0, 1) = 1$
Sum before scaling	8	5
Score after dividing by $\sqrt{2 + 2} = 2$	4	2.5

In the latent calculation, transform the content query as $\bar{q}^C = q^C W_{UK}^\top = (2, 3)$. Its dot products with $(1, 2)$ and $(2, 0)$ are still 8 and 4. This follows by writing the expanded product as $q^C (cW_{UK})^\top = (q^C W_{UK}^\top) c^\top$ for any row c. The rotary scores come from a separate dot product. The scale stays $\sqrt{d_h + d_R}$, where d_h is the expanded content width and d_R is the rotary width. Moving the key projection leaves the score represented by the dot product unchanged. In this example the latent and content widths happen to be equal.

In [15]

```
content_q, rotary_q, latent_rows, rotary_keys, key_maps, value_maps = (
    inputs
)
first_content = content_q[0]
first_rotary = rotary_q[0]
first_key_map = key_maps[0]
expanded_keys = latent_rows @ first_key_map
absorbed_query = first_content @ first_key_map.T

expanded_terms = first_content @ expanded_keys.T
absorbed_terms = absorbed_query @ latent_rows.T
rotary_terms = first_rotary @ rotary_keys.T
scale = np.sqrt(content_q.shape[-1] + rotary_q.shape[-1])
scaled_scores = (absorbed_terms + rotary_terms) / scale

np.testing.assert_array_equal(expanded_terms, [8, 4])
np.testing.assert_array_equal(absorbed_terms, expanded_terms)
np.testing.assert_array_equal(rotary_terms, [0, 1])
np.testing.assert_array_equal(scaled_scores, [4, 2.5])
print("Content from expanded keys", expanded_terms)
print("Content from latent rows  ", absorbed_terms)
print("Rotary contribution       ", rotary_terms)
print("Scaled scores             ", scaled_scores)
```

Output

```
Content from expanded keys [8. 4.]
Content from latent rows   [8. 4.]
Rotary contribution        [0. 1.]
Scaled scores              [4.  2.5]
```

Softmax of $(4, 2.5)$ gives first-head weights approximately $(0.81757448, 0.18242552)$. For example, the first probability is $e^4/(e^4 + e^{2.5}) = 1/(1 + e^{-1.5})$. Write these weights as a and b, so $a + b = 1$. The first head's value matrix is $\mathrm{diag}(1, 2)$, which sends the two latent rows to $(1, 4)$ and $(2, 0)$. Weighting these gives $(a + 2b, 4a)$. If we mix the latent rows first, we get $(a + 2b, 2a)$, which the value matrix again maps to $(a + 2b, 4a)$. A fixed linear projection distributes over the weighted sum.

The history consists of the third and fourth input arrays, which are `inputs[2]` and `inputs[3]` in Python. Each has two rows with two coordinates, so each has four scalars. Both heads share those histories. The sum is eight scalars. If we kept separate expanded content keys and values for both heads, that would take $2 \cdot 2 \cdot (2 + 2) = 16$ scalars, with another four for the shared rotary keys. This comparison counts the stored history. The current query and the learned projection matrices are separate.

In [16]

```
first_exp = np.exp(scaled_scores - scaled_scores.max())
first_weights = first_exp / first_exp.sum()
first_values = latent_rows @ value_maps[0]
weighted_values = first_weights @ first_values
projected_mixture = (first_weights @ latent_rows) @ value_maps[0]

np.testing.assert_allclose(first_weights, latent_w[0], atol=1e-12,
                           rtol=0)
np.testing.assert_allclose(weighted_values, latent_out[0], atol=1e-12,
                           rtol=0)
np.testing.assert_allclose(weighted_values, projected_mixture,
                           atol=1e-12, rtol=0)
with np.printoptions(precision=8):
    print("First-head weights", first_weights)
    print("Expand then mix   ", weighted_values)
    print("Mix then expand   ", projected_mixture)

history_scalars = inputs[2].size + inputs[3].size
assert history_scalars == 8
print("Latent history", inputs[2].shape, inputs[2].size, "scalars")
print("Rotary history", inputs[3].shape, inputs[3].size, "scalars")
print("Total history ", history_scalars, "scalars")
```

Output

```
First-head weights [0.81757448 0.18242552]
Expand then mix    [1.18242552 3.2702979 ]
Mix then expand    [1.18242552 3.2702979 ]
Latent history (2, 2) 4 scalars
Rotary history (2, 2) 4 scalars
Total history  8 scalars
```

9. Spend a cache budget (optional)

At the table's fixed dimensions, four KV heads use 48 MiB per 2,048-position sequence, and one uses 12 MiB. A 192 MiB payload budget fits $192/48 = 4$ or $192/12 = 16$ complete sequences, respectively.

With four KV heads, one extra position across 24 layers takes $24(4)(64 + 64)(2) = 24576$ bytes. Thus a single sequence can store

$$\frac{192 \cdot 2^{20}}{24576} = 8192 \text{ positions.}$$

These are storage limits only. The model must support that context length, and weights, workspaces, and unused reserved space need their own budgets. Packing multiple sequences does not let them attend to one another.

In [17]

```
budget = 192 * 2**20
for heads, expected in [(4, 4), (1, 16)]:
    per_sequence = cache_bytes(1, 24, 2048, heads, 64, 64, 2)
    assert budget // per_sequence == expected
    print(heads, "KV heads:", budget // per_sequence, "sequences")
per_position = cache_bytes(1, 24, 1, 4, 64, 64, 2)
assert budget // per_position == 8192
print("One sequence with four KV heads:", budget // per_position, "positions")
```

Output

```
4 KV heads: 4 sequences
1 KV heads: 16 sequences
One sequence with four KV heads: 8192 positions
```

10. Catch a position reset (optional)

Rotating $(1, 0)$ through angle θ gives $(\cos\theta, \sin\theta)$. The cached key at position 0 stays $(1, 0)$. The query at position 1 rotates through $\pi/2$ and becomes $(0, 1)$, so their unscaled dot product is zero. Resetting the query position to zero gives $(1, 0)$ and dot product one.

The key is unchanged in both cases; the new query's wrong position causes the error. Keeping the right cache rows is insufficient unless new queries and keys receive the correct absolute rotations. A contiguous cache starting at zero can use its length as the offset. A cache that evicts old positions must track the next logical position separately.

In [18]

```
def rotated_unit(position):
    angle = position * np.pi / 2
    return np.array([np.cos(angle), np.sin(angle)])
key = rotated_unit(0)
correct_dot = rotated_unit(1) @ key
reset_dot = rotated_unit(0) @ key
np.testing.assert_allclose([correct_dot, reset_dot], [0, 1], rtol=0, atol=1e-14)
print("Correct / reset:", round(correct_dot, 12), reset_dot)
```

Output

```
Correct / reset: 0.0 1.0
```

11. Choose the scale after absorption (optional)

The expanded content dot product uses four coordinates and the rotary term uses two. Together they are a dot product in six coordinates, scaled by $\sqrt{6}$. The key up-projection has shape 9×4. Moving it to the query side gives a transformed query of length nine:

$$q^C(cW)^\top = (q^C W^\top)c^\top.$$

The scalar numerator has not changed, so the denominator remains $\sqrt{6}$. Using $\sqrt{9+2} = \sqrt{11}$ would multiply all intended scores by $\sqrt{6/11} \approx 0.738549$. This preserves their order but shrinks their differences. For example, softmax of $(1, 0)$ assigns about 0.731059 to the first position; the extra scaling lowers it to about 0.676678. Equal scores remain uniform. Reassociation is an exact identity for fixed MLA maps in real arithmetic; the learned bottleneck itself is an architectural choice, not lossless compression of any arbitrary MHA model.

In [19]

```
from examples.ch04_attention.attention import stable_softmax
factor = np.sqrt(6 / 11)
original = stable_softmax(np.array([[1., 0.]]))[0]
wrong = stable_softmax(np.array([[factor, 0.]]))[0]
assert 0.5 < wrong[0] < original[0]
print("Correct weights:", original.round(6))
print("Wrong-scale weights:", wrong.round(6))
print("Score multiplier:", factor)
```

Output

```
Correct weights: [0.731059 0.268941]
Wrong-scale weights: [0.676678 0.323322]
Score multiplier: 0.7385489458759964
```

12. Design an inference comparison (optional)

This is a proposed protocol, not a measured benchmark. Compare the same causal decoder with identical weights, token IDs, positional conventions, head grouping, and evaluation settings. Disable dropout. Ensure any expert routing policy preserves prefix outputs when the grouping changes; a capacity rule that drops different tokens would confound the comparison.

First use a small decoder on CPU in float64 and token IDs `[7, 8, 9, 10, 7, 16]` (all within its vocabulary). Compare a full causal pass against chunks `(3, 2, 1)` and `(3, 1, 1, 1)`, with fresh caches. The two-row chunk has queries at positions 3 and 4; query 3 must not see key 4. Compare every vocabulary logit with absolute and relative tolerances of 10^{-10}, and report maximum errors by chunk. Also change a future input and check earlier outputs. If equality fails, inspect the first differing position, its mask, rotation, and cache contents before timing anything.

For timing, choose one NVIDIA A100 GPU, float16 inference, four prompts of length 512, and 128 fixed continuation IDs per sequence. Use a decoder with 12 layers, 12 query heads, three KV heads, key/value widths 64, and support for at least 640 positions. Record the actual software and attention kernels. Load weights and prepare identical replay inputs on the device beforehand. Project only the final hidden row to vocabulary logits in both timed paths.

Phase	Full-prefix path	Cached path
Prefill	Process all 512 inputs; return first continuation logits	Same prompt pass; also populate cache
Decode	Replay 127 inputs; recompute prefixes of lengths 513-639	Replay the same inputs; append one row per call

Prefill plus 127 append calls gives 128 predictions. The last supplied target ID need not be fed back. Replay fixes the same history even if roundoff would alter sampling. Include request-time allocation and any vocabulary projection in the appropriate phase; state whether token selection is timed. Do not include loading the model or copying prepared inputs from the host.

Warm up both paths. Run 20 paired trials, alternate their order, and synchronize the GPU before and after each timed phase. Report median and interquartile range for prefill and total decode time, decode time divided by 127, and $4(127)$ divided by decode seconds for aggregate throughput. The projected-array NumPy examples are correctness checks, not evidence for these GPU timings.

One stored position across the batch and all layers takes $4(12)(3)(64 + 64)(2) = 36864$ bytes. Cache payload is 18 MiB after prefill and $36864(639) = 23556096$ bytes, or 22.46484375 MiB, after the last timed call. Reserving 640 positions takes 22.5 MiB; report that separately. Recomputing retains no KV payload between calls but still constructs keys and values.

In separate profiling runs, record peak allocated and reserved device memory, with weights and inputs already resident as a baseline. Track cache and workspace allocations at the same instants: subtracting the final cache size from an earlier total peak does not recover peak temporary memory. Include old and new buffers that coexist during an append. Report payload, reserved cache, temporary workspace, and peak total distinctly. No speedup or quality winner is assumed before the experiment.

Checks

All twelve exercises have worked explanations. The numerical checks cover prediction counts, allowed pairs, head sharing, exact cache bytes, chunk positions, future-value invariance, storage rounding, and MLA reassociation. Numerical equality uses the same architecture and inputs; it is not a claim that MHA, GQA, and MLA have identical learned behavior.

Next Steps

Change one dimension or chunk boundary, predict the result, then rerun. For a full decoder, repeat the equality check on final vocabulary logits before measuring performance. Consult the chapter references on MQA, GQA, and DeepSeek-V2 for the original designs.